

Sistema Inteligente de

Reconocimiento Facial y Análisis de Probabilidades

Documentación completa del proyecto y del código

Institucion	SENATI
Equipo	Grupo 04
Stack	React + FastAPI + Supabase (Postgres/pgvector)
IA	InsightFace/ArcFace (DL) + scikit-learn (ML)
Repositorio	github.com/Grupo-04-Senati/Reconocimiento-Facial
Fecha	21 de septiembre de 2026

Contenido

- 1. Introduccion y objetivo
- 2. Que hace el sistema (resumen funcional)
- 3. Cumplimiento del PDF de requisitos
- 4. Deep Learning (DL) vs Machine Learning (ML) - concepto clave
- 5. Arquitectura general y stack tecnologico
- 6. Flujo completo de datos (pipeline paso a paso)
- 7. Modulos y paginas de la aplicacion
- 8. Explicacion del codigo - Backend (FastAPI)
- 9. Explicacion del codigo - Frontend (React)
- 10. Base de datos (Supabase / Postgres / pgvector)
- 11. Metricas y evaluacion del modelo
- 12. Seguridad y autenticacion
- 13. Despliegue (GitHub + Vercel + Railway + Supabase)
- 14. Limitaciones y trabajo futuro
- 15. Preguntas frecuentes para la defensa
- 16. Glosario de terminos

1. Introduccion y objetivo

Este proyecto es una **aplicacion web de reconocimiento facial** que ademas estima, mediante un modelo de aprendizaje automatico, la **probabilidad de que un rostro corresponda a un humano real** frente a un rostro generado o no valido (imagen de IA, dibujo, o cuando no se detecta una cara). El sistema combina dos familias de inteligencia artificial que trabajan en cadena:

- **Deep Learning (DL):** detecta el rostro y lo convierte en un vector numerico (embedding) que representa su identidad.
- **Machine Learning (ML):** a partir de variables numericas calculadas, entrega una probabilidad calibrada y una clasificacion (Humano Real / No Real), con metricas de calidad.

El objetivo academico es demostrar el ciclo completo: captura -> deteccion (DL) -> comparacion 1:N por similitud -> calculo de caracteristicas -> decision probabilistica (ML) -> registro, evaluacion y reentrenamiento; todo desplegado en la nube.

2. Que hace el sistema (resumen funcional)

- **Registrar rostros:** alta de personas con su embedding facial (una o varias fotos).
- **Reconocer (1:N):** dada una foto, busca la persona mas parecida en la base y devuelve quien es, con su similitud.
- **Comparar (1:1):** confronta dos rostros y calcula su similitud y probabilidad.
- **Clasificar Humano Real / No Real:** el ML entrega una probabilidad calibrada por cada caso.
- **Carga masiva etiquetada:** el administrador sube lotes de imagenes marcadas como Humano Real o No Real (IA).
- **Entrenar y evaluar el ML:** metricas accuracy, precision, recall, F1, FAR, FRR y matriz de confusion.
- **Historial y dashboard:** trazabilidad de cada reconocimiento y estadisticas globales.
- **Asistente (ChatBot):** explica el proyecto y guia sobre que fotos usar y como entrenar.

3. Cumplimiento del PDF de requisitos

La siguiente tabla mapea cada requisito del enunciado con el componente que lo implementa:

Requisito del enunciado	Como se cumple en el sistema
Reconocimiento facial	DL (InsightFace/ArcFace buffalo_s): deteccion + embedding 512-dim.
Analisis de probabilidades	ML (Regresion Logistica + calibracion): probabilidad calibrada por caso.
Comparacion 1:N y 1:1	Busqueda vectorial en pgvector (similitud coseno, indice HNSW).
Variables/caracteristicas	similitud, distancia, calidad de imagen e iluminacion.
Metricas de evaluacion	accuracy, precision, recall, F1, FAR, FRR, matriz de confusion.
Base de datos	Supabase (PostgreSQL) con extension pgvector.
Interfaz web	SPA en React 19 + TypeScript + Vite + Tailwind.

Requisito del enunciado	Como se cumple en el sistema
Despliegue en nube	Frontend en Vercel, backend en Railway, datos en Supabase.

4. Deep Learning (DL) vs Machine Learning (ML)

Es la distinción más importante del proyecto y la pregunta típica de defensa. **Se usan ambos, pero para cosas distintas**, y se conectan en cadena.

4.1 Deep Learning - la parte de visión (reconocer QUIEN)

- **Modelo:** InsightFace con ArcFace, paquete **buffalo_s** (redes neuronales convolucionales pre-entrenadas, ejecutadas con onnxruntime).
- **Función 1 - Detección:** localiza el rostro dentro de la imagen.
- **Función 2 - Embedding:** convierte el rostro en un **vector de 512 números** (normalizado) que representa la identidad. Rostros de la misma persona dan vectores cercanos.
- **Por qué DL:** aprender a representar un rostro a partir de píxeles requiere millones de imágenes y redes profundas; por eso se usa un modelo pre-entrenado, no reglas manuales.

4.2 Machine Learning - la parte de decisión (estimar la CONFIANZA)

- **Modelo:** **Regresión Logística** (scikit-learn) envuelta en **CalibratedClassifierCV** para que la probabilidad de salida sea realista (calibrada).
- **Entrada:** NO ve la imagen. Recibe **4 características numéricas**: similitud, distancia, calidad de imagen e iluminación.
- **Salida:** una **probabilidad calibrada** [0-1] de que sea Humano Real y la clasificación según el umbral de decisión.
- **Por qué ML clásico:** con pocas características y pocos datos, un modelo interpretable y ligero rinde mejor y se puede explicar (a diferencia de una red profunda).

Como se conectan DL y ML

DL produce los **números** (embedding -> similitud/distancia) y el pipeline mide la **calidad** e **iluminación** del rostro. Esos 4 números son la **entrada del ML**, que decide la probabilidad y la categoría. En una frase: **DL reconoce el rostro; ML decide cuanta confianza y a qué categoría pertenece.**

Aspecto	Deep Learning (DL)	Machine Learning (ML)
Librería	InsightFace / ArcFace	scikit-learn
Entrada	Imagen (píxeles)	4 variables numéricas
Salida	Embedding 512-dim	Probabilidad + clase
Tarea	Reconocer identidad	Clasificar / probabilizar
Se entrena aquí?	No (pre-entrenado)	Si (con tus datos)

5. Arquitectura general y stack tecnologico

Arquitectura de tres capas cliente-servidor con base de datos gestionada:

NAVEGADOR (usuario)	
React 19 + TypeScript + Vite + Tailwind	[Vercel]
v (HTTPS / JSON, token JWT)	
API REST -----	[Railway]
FastAPI (Python)	
-- Deep Learning : InsightFace/ArcFace (buffalo_s)	
-- Machine Learning : scikit-learn (LogReg + calibracion)	
v (SQL + RPC)	
SUPABASE (PostgreSQL + pgvector + Storage)	[Supabase]
-- personas / face_embeddings (VECTOR 512, HNSW)	
-- recognition_logs / ml_training_records	

5.1 Stack tecnologico (versiones reales)

Capa	Tecnologias principales
Frontend	React 19, TypeScript, Vite 8, Tailwind 4, React Router 7, Recharts, axios, react-webcam, lucide-react
Backend	FastAPI 0.115, Uvicorn, Pydantic 2, python-jose (JWT), python-multipart
Deep Learning	insightface 0.7.3, onnxruntime 1.19, opencv-python-headless, numpy
Machine Learning	scikit-learn 1.5.2, scipy, joblib
Base de datos	Supabase (PostgreSQL), extension pgvector 0.3.6, indice HNSW
Despliegue	Vercel (frontend), Railway (backend), GitHub (CI por push)

6. Flujo completo de datos (pipeline)

Recorrido de una imagen desde que entra hasta que se registra y alimenta al modelo:

#	Paso	Componente
1	El usuario captura con camara o sube una imagen. El frontend la redimensiona (max 800px, JPEG) para no saturar el backend.	Frontend
2	Se detecta el rostro y se calcula su calidad (nitidez = varianza del Laplaciano) e iluminacion (brillo), normalizadas [0,1].	face_service (DL+OpenCV)
3	Se genera el embedding de 512 dimensiones (ArcFace), normalizado.	embedding_service (DL)
4	Se busca en la base el vecino mas cercano por similitud coseno (busqueda 1:N).	vector_store + pgvector RPC
5	Con similitud, distancia, calidad e iluminacion, el ML calcula la probabilidad calibrada y decide la clase.	probability_service (ML)
6	Se muestra el resultado: quien es, probabilidad y clasificacion (Humano Real / No Real / No es humano).	FaceResultCard
7	Se registra en recognition_logs. Al confirmar, se crea un ml_training_records con la etiqueta verdadera.	audit_service

#	Paso	Componente
8	Entrenamiento: se juntan los registros etiquetados, se entrena y evalua el ML, y se recalculan metricas y umbral.	ml/trainer

7. Modulos y paginas de la aplicacion

Pagina	Funcion
Login	Autenticacion con JWT (roles admin / usuario).
Dashboard	Indicadores: total de personas, reconocimientos, estado del modelo.
Reconocimiento	Captura/sube foto, ejecuta 1:N y muestra identidad + clase. Avisa si falta registrar personas o entrenar el modelo.
Registro Facial	Registra una persona con 1 foto y permite confirmar manualmente si es Humano Real o No Real (genera dato de entrenamiento).
Seguridad (admin)	Carga masiva de imagenes con selector Humano Real / No Real (IA): cada carga registra, confirma, etiqueta y crea muestra de entrenamiento, todo en una accion.
Probabilidades	Comparacion 1:N y 1:1 mostrando las 7 variables (similitud, distancia, probabilidad, calidad, iluminacion, etc.) y la etiqueta Real/IA.
Entrenamiento ML	Entrena el modelo y muestra metricas, matriz de confusion, clasificacion y aviso de sobreajuste.
Historial	Lista de todos los reconocimientos con su resultado.

8. Explicacion del codigo - Backend (FastAPI)

Estructura backend/app/ organizada por responsabilidades: **core** (config/seguridad), **services** (logica), **ml** (entrenamiento y busqueda vectorial) y **api/routes** (endpoints REST).

8.1 core/ (configuracion y seguridad)

Archivo	Responsabilidad
config.py	Settings con Pydantic: claves de Supabase, JWT, y parametros del ML (ver 8.5).
security.py	Emision/validacion de JWT (HS256) y dependencias get_current_user / get_current_admin.
supabase_client.py	Clientes de Supabase (anon y service_role para operaciones de administrador).
logging_config.py	Logger centralizado.

8.2 services/ (logica de negocio)

Archivo	Responsabilidad
face_service.py	Deteccion del rostro (DL) y calculo de calidad (nitidez) e iluminacion.
embedding_service.py	Genera el embedding 512-dim y la funcion get_embedding_with_quality.
probability_service.py	Carga el modelo .joblib, predice la probabilidad calibrada, guarda el umbral de decision y dispara el reentrenamiento.
audit_service.py	Escribe recognition_logs y ml_training_records; lee el historial.
storage_service.py	Gestion de imagenes en Supabase Storage.

8.3 ml/ (aprendizaje automatico)

Archivo	Responsabilidad
trainer.py	Clase FaceTrainer: entrena LogReg + CalibratedClassifierCV, hace holdout 75/25, calcula metricas, la curva FAR/FRR y el aviso de overfitting, y guarda el modelo.
vector_store.py	Busqueda 1:N llamando al RPC match_face_embedding (pgvector + HNSW).

8.4 api/routes/ (endpoints REST)

Archivo	Endpoints principales
auth.py	Login y emision de token.
personas.py	Registrar persona (+embedding, tipo real/ia, calidad/iluminacion), listar, contar.
recognition.py	Reconocer (1:N), confirmar (crea dato de entrenamiento), auto-confirmar (exige modelo entrenado).
probabilities.py	Comparar 1:N y 1:1 con las 7 variables, info del modelo, prediccion manual (sliders).
models.py	Entrenar, recalcular, agregar-muestra (carga masiva unificada), estado del modelo.
dashboard.py / health.py	Estadisticas globales y verificacion de salud del servicio.

8.5 Parametros clave del ML (core/config.py)

```
UMBRAL_SIMILITUD = 0.40 # similitud coseno minima para aceptar coincidencia
MIN_TRAIN_PER_CLASS = 10 # minimo de ejemplos por clase para entrenar
UMBRAL_DECISION = 0.5 # probabilidad minima para clasificar Humano Real
# (el entrenador lo ajusta con la curva FAR/FRR)
```

Nota: subir UMBRAL_DECISION reduce el FAR (falsos aceptados) a costa de mas FRR; en biometria se prioriza un FAR bajo.

8.6 Ejemplo real - decision con el umbral (trainer.py)

```
# El modelo se entrena y la decision usa el umbral optimo:
y_pred = (y_prob >= self.umbra1_decision).astype(int)

# FAR/FRR: metricas biometricas clave
# FAR = FP / (FP + TN) -> falsos aceptados
# FRR = FN / (FN + TP) -> falsos rechazados
```

9. Explicacion del codigo - Frontend (React)

SPA en frontend/src/: **pages** (una por vista), **components** (reutilizables) y **services/api.ts** (cliente HTTP con axios que centraliza todas las llamadas al backend).

9.1 Componentes reutilizables

Componente	Funcion
CameraCapture	Captura de foto desde la webcam.
FaceResultCard	Tarjeta de resultado: identidad, probabilidad y clase (Humano Real / No Real / No es humano).
ProbabilityChart	Grafico de barras/probabilidades (Recharts).
SimilarityBar	Barra visual de similitud.
ChatBot	Asistente que explica el proyecto y guia sobre que fotos usar y como entrenar.
Sidebar / Header / Card / StatCard	Layout, navegacion y tarjetas de UI.
ProtectedRoute	Protege rutas segun sesion/rol.

9.2 services/api.ts (cliente HTTP)

Centraliza las llamadas: login, reconocer, confirmar, registrarPersona, **agregarMuestra** (carga masiva), entrenar, obtenerMetricas, personasCount, comparaciones 1:N y 1:1. Adjunta el token JWT en cada peticion.

9.3 Optimizacion importante: redimension en cliente

Antes de enviar, las paginas reducen la imagen (canvas -> JPEG, max 800px) y convierten formatos como AVIF/WEBP. Esto evito errores 422/timeout del backend con imagenes pesadas y acelero la carga masiva.

10. Base de datos (Supabase / PostgreSQL / pgvector)

Cuatro tablas y una funcion de busqueda vectorial. La extension **pgvector** permite almacenar el embedding de 512 dimensiones y buscar por similitud coseno con un indice **HNSW** (rapido en grandes volúmenes).

Tabla	Columnas clave	Rol
personas	id, nombre, email, activo, tipo (real/ia)	Identidades registradas y su etiqueta manual Real/IA.
face_embeddings	persona_id, embedding VECTOR(512), modelo	Vector facial (DL) por persona; indice HNSW coseno.
recognition_logs	similitud, distancia, umbral, coincide, probabilidad_calibrada, calidad_imagen, iluminacion	Traza de cada reconocimiento (historial).

Tabla	Columnas clave	RoI
ml_training_records	similitud, calidad_imagen, iluminacion, distancia, resultado_real	Dataset etiquetado que entrena al ML.

10.1 Funcion de busqueda 1:N (RPC match_face_embedding)

```
SELECT fe.id, fe.persona_id, p.nombre,
       1 - (fe.embedding <=> query_embedding) AS similitud,
       (fe.embedding <=> query_embedding) AS distancia
FROM face_embeddings fe
JOIN personas p ON p.id = fe.persona_id
WHERE p.activo = TRUE
      AND 1 - (fe.embedding <=> query_embedding) > match_threshold
ORDER BY fe.embedding <=> query_embedding -- <=> = distancia coseno
LIMIT match_count;
```

El operador <=> de pgvector calcula la distancia coseno; la similitud es 1 - distancia. El índice HNSW acelera esta búsqueda.

Migraciones aplicadas: 000-010 (base), 011 (calidad/iluminacion en logs) y 012 (columna tipo real/ia en personas).

11. Metricas y evaluacion del modelo

El ML se evalua con metricas estandar de clasificacion y biometria. Con datos suficientes (≥ 10 registros y ≥ 4 por clase) se usa **holdout 75/25** (se entrena con el 75% y se mide con el 25% no visto); si no, se avisa que las metricas no son fiables.

Metrica	Formula	Que mide
Accuracy	$(TP+TN)/total$	Aciertos globales.
Precision	$TP/(TP+FP)$	De los marcados Real, cuantos lo eran.
Recall	$TP/(TP+FN)$	De los Reales, cuantos detecto.
F1-score	media armonica	Equilibrio precision/recall.
FAR	$FP/(FP+TN)$	Falsos aceptados (critico en biometria).
FRR	$FN/(FN+TP)$	Falsos rechazados.

11.1 Umbral de decision y curva FAR/FRR

El entrenador prueba umbrales del 5% al 95% y elige el que **minimiza el FAR** manteniendo el FRR en un rango razonable (evita el caso de rechazar a todos). Asi se prioriza no aceptar falsos, como recomienda la biometria.

11.2 Aviso de sobreajuste (overfitting)

Si el modelo llega a 100% con menos de 50 registros, el sistema muestra un aviso honesto: con pocos datos las clases se separan demasiado facil. El aviso recomienda subir a 100-200 imagenes por clase, variadas, e incluir casos dificiles (rostros reales con sombras/baja resolucion y caras de IA de distintos motores). Es una senal de rigor, no un error.

12. Seguridad y autenticacion

- **JWT (HS256)** con expiracion de 60 minutos; el token viaja en la cabecera Authorization (Bearer).
- **Roles:** administrador (entrenar, cargar dataset, gestionar) y usuario (reconocer). Dependencias `get_current_admin / get_current_user`.
- **RLS** (Row Level Security) en la tabla de usuarios de Supabase.
- **Buenas practicas:** no se exponen datos sensibles en la URL; las claves viven en variables de entorno.

13. Despliegue (GitHub + Vercel + Railway + Supabase)

- **GitHub** (Grupo-04-Senati/Reconocimiento-Facial, rama main): repositorio central; cada push dispara el despliegue.
- **Vercel:** aloja el frontend. Build: `tsc -b && vite build`. Auto-deploy en cada push.
- **Railway:** aloja el backend FastAPI (uvicorn). Auto-deploy en cada push.
- **Supabase:** base de datos PostgreSQL + pgvector + Storage. Las migraciones SQL se aplican en su editor.

Recordatorio operativo

Tras cada despliegue conviene recargar con **Ctrl+Shift+R** para evitar cache. Las migraciones 011 y 012 deben estar aplicadas en Supabase para que se guarden calidad/iluminacion y la etiqueta real/ia.

14. Limitaciones y trabajo futuro

- **Dataset pequeno/experimental:** con pocos datos el ML sobreajusta (100%); la solución real es más datos variados.
- **Real vs IA no es detección de deepfakes:** la distinción se apoya en señales indirectas (similitud, calidad, iluminación) más una etiqueta manual. Para producción se necesitaría un módulo dedicado de anti-spoofing / liveness.
- **buffalo_s es un modelo ligero:** prioriza velocidad; un modelo mayor (buffalo_l) daría embeddings más precisos a mayor costo.
- **Trabajo futuro:** ampliar el dataset, agregar detección de vida (parpadeo/textura), validación cruzada k-fold y más características.

15. Preguntas frecuentes para la defensa

El proyecto usa DL o ML?

Ambos. DL (ArcFace) reconoce el rostro generando el embedding; ML (Regresión Logística) decide la probabilidad de Humano Real y la categoría. Se conectan: la salida de DL es la entrada de ML.

Por qué las métricas dan 100%?

Porque hay pocos datos y las clases se separan fácil (sobreajuste). El sistema lo detecta y lo avisa. Con más datos variados baja a valores realistas (85-93%).

Como reduces el FAR?

Subiendo el umbral de decisión. El entrenador recorre la curva FAR/FRR y elige el umbral que minimiza el FAR sin rechazar a todos.

Detecta deepfakes o imágenes de IA de verdad?

No de forma robusta. Usa señales indirectas y una etiqueta manual; es una aproximación educativa. La detección real requiere anti-spoofing dedicado.

Que pasa si la foto no tiene rostro?

El backend responde 'No es humano' (no se detectó cara) y no lo cuenta como coincidencia.

Por qué Regresión Logística y no una red neuronal?

Con 4 características y pocos datos, un modelo interpretable y ligero rinde mejor, se explica fácil y su probabilidad se calibra bien.

Que es un embedding?

Un vector de 512 números que resume la identidad del rostro; rostros parecidos dan vectores cercanos (menor distancia coseno).

16. Glosario de términos

Término	Definición
Embedding	Vector de 512 números que representa un rostro (salida del DL).
Similitud coseno	Medida de parecido entre dos vectores (1 = idénticos).
1:N	Comparar un rostro contra todos los registrados (identificación).

Termino	Definicion
1:1	Comparar dos rostros entre si (verificacion).
Calibracion	Ajuste para que la probabilidad de salida sea realista.
FAR / FRR	Tasa de falsos aceptados / falsos rechazados.
Holdout	Reservar parte de los datos para evaluar sin hacer trampa.
Overfitting	El modelo memoriza los pocos datos y no generaliza.
pgvector / HNSW	Extension y algoritmo de indice para busqueda vectorial rapida.
JWT	Token firmado que autentica al usuario en cada peticion.

Documento generado para la sustentacion academica del proyecto - SENATI, Grupo 04.